KPR Institute of Engineering and Technology
(Autonomous)
Avinashi Road, Arasur, Coimbatore - 641 407

LEARN BEYOND

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

LABORATORY MANUAL

U25CSG13 - Object Oriented Programming Using Java Laboratory

VISION AND MISSION OF THE INSTITUTION

Vision

To become a premier institute of academic excellence by imparting technical, intellectual and professional skills to students for meeting the diverse needs of industry, society, the nation and the world at large.

Mission

- Commitment to offer value-based education and enhancement of practical skills.
- Continuous assessment of teaching and learning processes through scholarly activities.
- Enriching research and innovation activities in collaboration with industry and institutes of repute.
- Ensuring the academic processes to uphold culture, ethics and social responsibilities.

VISION AND MISSION OF THE DEPARTMENT

Vision

To be a premier centre for education, dissemination of knowledge and research in the frontier areas of computer science and engineering to serve the community with moral values.

Mission

- Provide holistic education incorporating the state-of-the-art technologies to produce successful professionals.
- Facilitate the students to pursue higher education and research in the areas related to Computer Science and Engineering.
- Promote strong collaborations with the industries and steer the students to nurture their interest in continuous learning to meet the changing needs of the society.

Program Educational Objectives

PEO 1: Acquire knowledge and skills on cutting edge technologies in the field of computer science and engineering.

PEO 2: Inculcate a passion for continuous learning through further studies and research in the field of computer science and engineering.

PEO 3: Develop innovative ideas upholding the rich value systems to solve the changing technological needs.

Program Outcomes (POs)

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions (WK8)

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment (WK1, WK5, and WK7)

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change (WK8) Knowledge and Attitude Profile (WK)

Program Specific Outcomes

PSO 1: Solve complex engineering problems by building integrated systems across various

domains of Computational Intelligence.

PSO 2: Apply technical skills to produce solutions that meet the requirements of the

emerging technologies in Computer Science and Engineering.

Course Outcomes (COs)

CO 1: Describe object-oriented programming concepts and Java fundamentals for developing simple applications (**Understand)**

CO 2: Develop Java applications using inheritance, polymorphism, abstract classes, interfaces, and packages (**Apply)**

CO 3: Develop Java programs incorporating exception handling and multithreading concepts (**Apply)**

CO 4: Develop Java applications using collections, generics, and related utility classes (**Apply)**

CO 5: Develop Java applications using file handling, stream processing, and object serialization techniques (**Apply)**

Rubrics for Assessment

	Criteria	Excellent (4 Marks)	Good (3 Marks)	Adequate (2 Mark)	Inadequate (1 Mark)
A. Preparation & Observation	**Criteria #1** Ability to setup and conduct experiments	Able to develop contingency or alternative plans and anticipate problems during experiment.	Able to develop contingency or alternative plans.	Able to use theoretical framework, measurement techniques, testing apparatus or model.	Unable to identify theoretical framework, measurement techniques, testing apparatus or model.
	Criteria #2 Ability to take measurements / readings and present data	Able to formulate, controls and evaluate alternatives of the experiment. Able to evaluate data and relate to engineering phenomena for decision-making.	Able to evaluate data and relate to engineering phenomena for decision-making.	Able to apply constraint and assumption into the experimental design. Able to conduct experiment correctly and collect data.	Unable to discuss experimental processes and protocols.
B. Results & Interpretation	**Criteria #3** Ability to analyze the data theoretically and logically to conclude experimental results	Able to combine /organize more than one set of data, interpret data and make meaningful conclusion.	Able to evaluate or compare data and make meaningful conclusion.	Able to select and use and apply appropriate techniques or methods to analyze the data.	Unable to select and describe the techniques or methods of analyzing the data.
	Criteria #4 Ability to interpret and discuss any discrepancies between theoretical and experimental results	Able to verify and/or validate several sets of data and relates to engineering phenomena for decision making.	Able to verify and/or validate data and relate to engineering phenomena for decision making.	Able to identify and verify how results relate/differ from theory or previous results.	Unable to identify how results relate/differ from theory or previous results.
C. Viva-Voce	**Criteria #5** Demonstrate the ability to respond effectively to questions	Able to listen carefully and respond to questions appropriately; is able to explain and interpret results to the teacher.	Able to listen carefully and respond to questions appropriately.	Misunderstand the questions and does not respond appropriately to the teacher, or has some trouble in answering questions.	Unable to listen carefully to questions and does not provide an appropriate answer, or is unable to answer questions.

INDEX

S. No.	Name of the Experiment
1	Design and Implementation of Java Programs Demonstrating Classes, Objects, Constructors, and Methods
2	Development of Applications Illustrating Static and Non-static Members, Access Modifiers, Abstraction, and Encapsulation
3	Implementation of Console-based Applications Using Scanner and System Classes for Input and Output Processing
4	Design and Implementation of Applications Demonstrating Inheritance and Its Various Types
5	Development of Applications Illustrating Compile-time and Runtime Polymorphism Using Method Overloading and Method Overriding
6	Implementation of Applications Using Abstract Classes, Interfaces, and Dynamic Method Dispatch Mechanisms
7	Design and Development of Modular Java Applications Using Built-in and User-defined Packages
8	Implementation of Robust Exception Handling Mechanisms Using Built-in and User-defined Exceptions
9	Development of Multithreaded Applications Using Thread Class and Runnable Interface
10	Design and Implementation of Synchronized Multithreaded Applications with Inter-thread Communication
11	Development of Applications Demonstrating Thread Lifecycle Management and Deadlock Prevention Techniques
12	Implementation of Generic Collection-based Applications Using List, Set, Map, and Queue Interfaces
13	Design of Applications Utilizing Comparator and Comparable Interfaces for Customized Sorting Operations
14	Development of File Processing Applications Using Java I/O Streams, Readers, Writers, and Buffered Streams
15	Implementation of Object Persistence Mechanisms Using Serialization and Deserialization Techniques

1. Design and implementation of Java programs demonstrating classes, objects, constructors, and methods

Aim

To design and implement a Java program for a Library Management System using classes, objects, constructors, and methods.

Algorithm / Procedure

Step 1: Start the program.

Step 2: Create a class named Book.

Step 3: Declare the instance variables bookId, title, author, category, price, and available.

Step 4: Create a parameterized constructor to initialize all the instance variables.

Step 5: Create a method named displayBookDetails() to display the book information.

Step 6: Create the Main class containing the main() method.

Step 7: Create two Book objects by passing the required values to the constructor.

Step 8: Invoke the displayBookDetails() method using both objects.

Step 9: Display the details of the books.

Step 10: Stop the program.

Program

```
class Book {

        int bookId;
        String title, author, category;
        double price;
        boolean available;

        Book(int bookId, String title, String author, String category, double price, boolean
                 available){
                this.bookId=bookId;
                this.title=title;
                this.author=author;
                this.category=category;
                this.price=price;
                this.available=available;
        }
        void displayBookDetails(){
                System.out.println("Book ID       : "+bookId);
                System.out.println("Title         : "+title);
                System.out.println("Author        : "+author);
                System.out.println("Category      : "+category);
```

```
                System.out.println("Price          : ₹"+price);
                System.out.println("Availability   : "+(available?"Available":"Not
                                                                    Available"));
    }
}
public class Main{
        public static void main(String[] args){
                Book b1=new Book(101,"Java Programming","Herbert
                                        Schildt","Programming",650.00,true);
                Book b2=new Book(102,"Database Systems","Raghu
                                        Ramakrishnan","Database",720.00,false);
                b1.displayBookDetails();
                b2.displayBookDetails();
        }
}
```

Output

```
Book ID       : 101
Title         : Java Programming
Author        : Herbert Schildt
Category      : Programming
Price         : ₹650.0
Availability  : Available

Book ID       : 102
Title         : Database Systems
Author        : Raghu Ramakrishnan
Category      : Database
Price         : ₹720.0
Availability  : Not Available
```

Viva Questions

1. What is a class in Java?
2. What is the purpose of a constructor?
3. What is the difference between an object and a class?
4. What is the difference between a constructor and a method?

Result

Thus, the Java program demonstrating the concepts of classes, objects, constructors, and methods through a Library Management System was successfully designed, implemented, and verified with the expected output.

2. Development of applications illustrating static and non-static members, access modifiers, abstraction, and encapsulation

Aim

To design and implement a Java program for an Employee Management System using static and non-static members, access modifiers, abstraction, and encapsulation.

Algorithm / Procedure

Step 1: Start the program.

Step 2: Create an abstract class named Employee.

Step 3: Declare a static variable company to represent the common company name.

Step 4: Declare non-static variables for employee details using different access modifiers:

public – department

private – salary

protected – designation

package-private (default) – location

Step 5: Create a parameterized constructor to initialize the employee details.

Step 6: Implement getter and setter methods to access and modify the private variable salary, demonstrating encapsulation.

Step 7: Declare an abstract method displayRole() in the abstract class.

Step 8: Create a subclass Developer that extends the Employee class and provides the implementation for the abstract method.

Step 9: Create an object of the Developer class and invoke the methods to display the employee details and role.

Step 10: Stop the program.

Program

```
abstract class Employee {
    // Static Member
    static String company = "ABC Technologies";
    // Non-static Members
    int empId;
    String empName;
    // Access Modifiers
    public String department;
    protected String designation;
    String location;            // Default Access Modifier
    private double salary;
    // Constructor
    Employee(int empId, String empName, String department,
            String designation, String location, double salary) {
        this.empId = empId;
        this.empName = empName;
        this.department = department;
        this.designation = designation;
        this.location = location;
        this.salary = salary;
    }
    // Encapsulation - Setter Method
    public void setSalary(double salary) {
        this.salary = salary;
```

```
    }
    // Encapsulation - Getter Method
    public double getSalary() {
        return salary;
    }
    // Non-static Method
    void displayEmployeeDetails() {
        System.out.println("\nEmployee Details");
        System.out.println("----------------------------");
        System.out.println("Company      : " + company);
        System.out.println("Employee ID  : " + empId);
        System.out.println("Employee Name: " + empName);
        System.out.println("Department   : " + department);
        System.out.println("Designation  : " + designation);
        System.out.println("Location     : " + location);
        System.out.println("Salary       : " + getSalary());
    }
    // Abstract Method
    abstract void displayRole();
}
// Concrete Class
class Developer extends Employee {
    Developer(int empId, String empName, String department,
            String designation, String location, double salary) {
        super(empId, empName, department,
```

```
            designation, location, salary);
    }
    @Override
    void displayRole() {
        System.out.println("Role         : Java Developer");
    }
}
public class Main {
    public static void main(String[] args) {
        Developer emp = new Developer(
                101,
                "Rahul",
                "Software Development",
                "Developer",
                "Chennai",
                55000);
        emp.displayEmployeeDetails();
        emp.displayRole();
    }
}
```

Output

```
Employee Details
----------------------------
Company      : ABC Technologies
Employee ID  : 101
Employee Name: Rahul
```

Department : Software Development

Designation : Developer

Location : Chennai

Salary : 55000.0

Role : Java Developer

Viva Questions

1. What is the difference between a static member and a non-static member in Java?
2. Name the four access modifiers in Java and explain their accessibility.
3. What is abstraction? How is it implemented in this program?
4. What is encapsulation? Why are getter and setter methods used?

Result

Thus, the Java program for an Employee Management System demonstrating static and non-static members, access modifiers, abstraction, and encapsulation was successfully designed, implemented, and verified with the expected output.

3. Implementation of console-based applications using Scanner and System classes for input and output processing

Aim

To design and implement a Java program for a Student Admission Management System using the Scanner and System classes for input and output processing.

Algorithm / Procedure

Step 1: Start the program.

Step 2: Import the java.util.Scanner package.

Step 3: Create a Scanner object to receive input from the user.

Step 4: Read the student details such as Student ID, Student Name, Department, Age, and Percentage using the Scanner class.

Step 5: Display the entered student details using the System.out.println() method.

Step 6: Close the Scanner object.

Step 7: Stop the program.

Program

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Student ID: ");

        int studentId = sc.nextInt();

        sc.nextLine();

        System.out.print("Enter Student Name: ");

        String studentName = sc.nextLine();

        System.out.print("Enter Department: ");
```

```
        String department = sc.nextLine();
        System.out.print("Enter Age: ");
        int age = sc.nextInt();
        System.out.print("Enter Percentage: ");
        double percentage = sc.nextDouble();
        System.out.println("\nStudent Admission Details");
        System.out.println("----------------------------");
        System.out.println("Student ID      : " + studentId);
        System.out.println("Student Name    : " + studentName);
        System.out.println("Department      : " + department);
        System.out.println("Age             : " + age);
        System.out.println("Percentage      : " + percentage + "%");
        sc.close();
    }
}
```

Output

```
Enter Student ID: 101
Enter Student Name: Rahul Sharma
Enter Department: Computer Science and Engineering
Enter Age: 19
Enter Percentage: 89.5

Student Admission Details
----------------------------
Student ID      : 101
```

Student Name : Rahul Sharma

Department : Computer Science and Engineering

Age : 19

Percentage : 89.5%

Viva Questions

1. What is the purpose of the Scanner class in Java?
2. What is the difference between next() and nextLine() methods of the Scanner class?
3. What is the purpose of System.in and System.out in Java?
4. Why is sc.nextLine() used after nextInt() in this program?

Result

Thus, the Java program for a Student Admission Management System using the Scanner and System classes for input and output processing was successfully designed, implemented, and verified with the expected output.

4. Design and implementation of applications demonstrating inheritance and its various types

Aim

To design and implement a Java program for a Banking System demonstrating the various types of inheritance in Java.

Algorithm / Procedure

Step 1: Start the program.

Step 2: Create a parent class named Account containing the account holder's name and account number.

Step 3: Create a child class SavingsAccount that inherits the properties of the Account class.

Step 4: Create another child class CurrentAccount that also inherits the properties of the Account class.

Step 5: Create a class PremiumSavingsAccount that extends SavingsAccount to demonstrate multilevel inheritance.

Step 6: Create objects for SavingsAccount, CurrentAccount, and PremiumSavingsAccount.

Step 7: Invoke the inherited methods to display the account details.

Step 8: Display the output.

Step 9: Stop the program.

Program

```
// Parent Class

class Account {

   String accountHolder;

   int accountNumber;

   Account(String accountHolder, int accountNumber) {

      this.accountHolder = accountHolder;

      this.accountNumber = accountNumber;
```

```
    }
    void displayAccount() {
        System.out.println("Account Holder : " + accountHolder);
        System.out.println("Account Number : " + accountNumber);
    }
}
// Single Inheritance
class SavingsAccount extends Account {
    double interestRate;
    SavingsAccount(String accountHolder, int accountNumber,
                double interestRate) {
        super(accountHolder, accountNumber);
        this.interestRate = interestRate;
    }
    void displaySavings() {
        displayAccount();
        System.out.println("Interest Rate  : " + interestRate + "%");
    }
}
// Multilevel Inheritance
class PremiumSavingsAccount extends SavingsAccount {
    String premiumBenefits;
    PremiumSavingsAccount(String accountHolder,
                        int accountNumber,
                        double interestRate,
```

```
                    String premiumBenefits) {
        super(accountHolder, accountNumber, interestRate);
        this.premiumBenefits = premiumBenefits;
    }
    void displayPremium() {
        displaySavings();
        System.out.println("Benefits       : " + premiumBenefits);
    }
}
// Hierarchical Inheritance
class CurrentAccount extends Account {
    double overdraftLimit;
    CurrentAccount(String accountHolder,
                   int accountNumber,
                   double overdraftLimit) {
        super(accountHolder, accountNumber);
        this.overdraftLimit = overdraftLimit;
    }
    void displayCurrent() {
        displayAccount();
        System.out.println("Overdraft Limit: ₹" + overdraftLimit);
    }
}
public class Main {
    public static void main(String[] args) {
```

```
        System.out.println("Single Inheritance");
        SavingsAccount s1 =
                new SavingsAccount(
                        "Rahul",
                        1001,
                        6.5);
        s1.displaySavings();
        System.out.println("\nMultilevel Inheritance");
        PremiumSavingsAccount p1 =
                new PremiumSavingsAccount(
                        "Ananya",
                        1002,
                        7.0,
                        "Free Insurance");
        p1.displayPremium();
        System.out.println("\nHierarchical Inheritance");
        CurrentAccount c1 =
                new CurrentAccount(
                        "Karthik",
                        1003,
                        50000);
        c1.displayCurrent();
    }
}
```

Output

Single Inheritance

Account Holder : Rahul

Account Number : 1001

Interest Rate : 6.5%

ultilevel Inheritance

Account Holder : Ananya

Account Number : 1002

Interest Rate : 7.0%

Benefits : Free Insurance

Hierarchical Inheritance

Account Holder : Karthik

Account Number : 1003

Overdraft Limit: ₹50000.0

Viva Questions

1. What is inheritance in Java?
2. Name the types of inheritance supported by Java classes.
3. What is the purpose of the super keyword in inheritance?
4. What is the difference between single, multilevel, and hierarchical inheritance?

Result

Thus, the Java program for a Banking System demonstrating single, multilevel, and hierarchical inheritance was successfully designed, implemented, and verified with the expected output.

5. Development of applications illustrating compile-time and runtime polymorphism using method overloading and method overriding

Aim

To design and implement a Java program for an Online Payment System demonstrating compile-time and runtime polymorphism using method overloading and method overriding.

Algorithm / Procedure

Step 1: Start the program.

Step 2: Create a class named Payment.

Step 3: Implement method overloading by creating multiple makePayment() methods with different parameter lists.

Step 4: Create a subclass UPIPayment that extends the Payment class.

Step 5: Override the makePayment(double amount) method in the subclass to demonstrate runtime polymorphism.

Step 6: Create an object of the UPIPayment class.

Step 7: Invoke the overloaded methods to demonstrate compile-time polymorphism.

Step 8: Invoke the overridden method using the subclass object to demonstrate runtime polymorphism.

Step 9: Display the payment details.

Step 10: Stop the program.

Program

```
class Payment {
    // Method Overloading (Compile-time Polymorphism)
    void makePayment(double amount) {
        System.out.println("Payment Amount : ₹" + amount);
    }
    void makePayment(double amount, String paymentMode) {
        System.out.println("Payment Amount : ₹" + amount);
        System.out.println("Payment Mode   : " + paymentMode);
    }
}
// Subclass
class UPIPayment extends Payment {
    // Method Overriding (Runtime Polymorphism)
    @Override
    void makePayment(double amount) {
        System.out.println("UPI Payment Successful");
        System.out.println("Amount Paid : ₹" + amount);
    }
}
public class Main {
    public static void main(String[] args) {
        UPIPayment payment = new UPIPayment();
        System.out.println("Compile-time Polymorphism");
        payment.makePayment(1500.00, "Credit Card");
```

```
        System.out.println("\nRuntime Polymorphism");
        payment.makePayment(2000.00);
    }
}
```

Output

Compile-time Polymorphism

Payment Amount : ₹1500.0

Payment Mode : Credit Card

Runtime Polymorphism

UPI Payment Successful

Amount Paid : ₹2000.0

Viva Questions

1. What is polymorphism in Java?
2. What is the difference between compile-time and runtime polymorphism?
3. What is method overloading? Give an example.
4. What is method overriding? Why is the @Override annotation used?

Result

Thus, the Java program for an Online Payment System demonstrating compile-time and runtime polymorphism using method overloading and method overriding was successfully designed, implemented, and verified with the expected output.

6. Implementation of applications using abstract classes, interfaces, and dynamic method dispatch mechanisms

Aim

To design and implement a Java program for a Smart Home Appliance System using abstract classes, interfaces, and dynamic method dispatch.

Algorithm / Procedure

Step 1: Start the program.

Step 2: Create an interface named RemoteControl with the methods turnOn() and turnOff().

Step 3: Create an abstract class named Appliance containing an abstract method displayAppliance().

Step 4: Create a class SmartTV that extends the Appliance class and implements the RemoteControl interface.

Step 5: Provide implementations for all the abstract and interface methods.

Step 6: Create a parent class reference of type Appliance and assign it to a SmartTV object to demonstrate dynamic method dispatch.

Step 7: Invoke the methods using the parent class reference.

Step 8: Display the appliance details and status.

Step 9: Stop the program.

Program

```
// Interface
interface RemoteControl {
    void turnOn();
    void turnOff();
}
// Abstract Class
abstract class Appliance {
    abstract void displayAppliance();
}
// Concrete Class
class SmartTV extends Appliance implements RemoteControl {
    @Override
    void displayAppliance() {
        System.out.println("Appliance : Smart TV");
    }
    @Override
    public void turnOn() {
        System.out.println("Smart TV is Turned ON");
    }
    @Override
    public void turnOff() {
        System.out.println("Smart TV is Turned OFF");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        // Dynamic Method Dispatch
        Appliance appliance = new SmartTV();
        appliance.displayAppliance();
        RemoteControl remote = new SmartTV();
        remote.turnOn();
        remote.turnOff();
    }
}
```

Output

```
Appliance : Smart TV
Smart TV is Turned ON
Smart TV is Turned OFF
```

Viva Questions

1. What is an abstract class in Java?
2. What is an interface?
3. Why do we use the implements keyword in Java?
4. What is the difference between an abstract class and an interface?

Result

Thus, the Java program for a Smart Home Appliance System using abstract classes, interfaces, and dynamic method dispatch was successfully designed, implemented, and verified with the expected output.

7. Design and Development of Modular Java Applications Using Built-in and User-defined Packages

Aim

To design and develop a modular Java application using built-in and user-defined packages for organizing classes and promoting code reusability.

Program

(Inside package employee)

File 1: Employee.java

```
package employee;

public class Employee {

    private int empId;

    private String empName;

    private double salary;


    public Employee(int empId, String empName, double salary) {

        this.empId = empId;

        this.empName = empName;

        this.salary = salary;

    }


    public void displayEmployee() {

        System.out.println("\nEmployee Details");

        System.out.println("--------------------------");

        System.out.println("Employee ID   : " + empId);

        System.out.println("Employee Name : " + empName);

        System.out.println("Salary        : " + salary);
```

```
    }
}
```

File 2: Main.java

```
import java.util.Scanner;
import employee.Employee;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Employee ID : ");
        int id = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Employee Name : ");
        String name = sc.nextLine();

        System.out.print("Enter Salary : ");
        double salary = sc.nextDouble();

        Employee emp = new Employee(id, name, salary);

        emp.displayEmployee();
```

```
        sc.close();
    }
}
```

Folder Structure

```
PackageDemo
│
├── employee
│       Employee.java
│
└── Main.java
```

Compilation

```
javac -d . Employee.java
javac Main.java
java Main
```

Output

```
Enter Employee ID : 101
Enter Employee Name : Rahul Sharma
Enter Salary : 55000


Employee Details
---------------------------
Employee ID   : 101
Employee Name : Rahul Sharma
Salary        : 55000.0
```

Output

Result

Thus, a modular Java application using built-in and user-defined packages was designed and implemented successfully.

Viva Questions

1. What is a package in Java?
2. What is the difference between built-in and user-defined packages?
3. How do you create a package in Java?
4. Why are packages used in Java applications?
5. What is the purpose of the import statement?

8. Implementation of Robust Exception Handling Mechanisms Using Built-in and User-defined Exceptions

Aim

To implement robust exception handling mechanisms in Java using built-in and user-defined exceptions.

Program

```
import java.util.InputMismatchException;

import java.util.Scanner;

// User-defined Exception

class InsufficientBalanceException extends Exception {

    public InsufficientBalanceException(String message) {

        super(message);

    }

}

// Bank Account Class

class BankAccount {

    private double balance;

    public BankAccount(double balance) {

        this.balance = balance;

    }

    public void withdraw(double amount)

            throws InsufficientBalanceException {

        if (amount > balance) {

            throw new InsufficientBalanceException(

                    "Withdrawal failed! Insufficient balance.");

        }
```

```
        balance -= amount;
        System.out.println("Withdrawal Successful.");
        System.out.println("Remaining Balance : Rs. " + balance);
    }
}
// Main Class
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter Initial Balance : ");
            double balance = sc.nextDouble();
            BankAccount account = new BankAccount(balance);
            System.out.print("Enter Withdrawal Amount : ");
            double amount = sc.nextDouble();
            account.withdraw(amount);
        }
        // Built-in Exception
        catch (InputMismatchException e) {
            System.out.println("Error: Please enter numeric values only.");
        }
        // User-defined Exception
        catch (InsufficientBalanceException e) {
            System.out.println("Error: " + e.getMessage());
        }
```

```
        finally {
            System.out.println("Transaction Completed.");
            sc.close();
        }
    }
}
```

Output 1 (Successful Transaction)

Enter Initial Balance : 10000

Enter Withdrawal Amount : 2500

Withdrawal Successful.

Remaining Balance : Rs. 7500.0

Transaction Completed.

Output 2 (User-defined Exception)

Enter Initial Balance : 5000

Enter Withdrawal Amount : 8000

Error: Withdrawal failed! Insufficient balance.

Transaction Completed.

Output 3 (Built-in Exception)

Enter Initial Balance : abc

Error: Please enter numeric values only.

Transaction Completed.

Result

Thus, exception handling using built-in and user-defined exceptions was implemented successfully.

Viva Questions

1. What is an exception?
2. Differentiate checked and unchecked exceptions.
3. What is the purpose of the finally block?
4. How do throw and throws differ?
5. How do you create a user-defined exception?

9. Development of Multithreaded Applications Using Thread Class and Runnable Interface

Aim

To develop multithreaded Java applications using the Thread class and Runnable interface.

Program

```
// Thread using Thread class
class ReservationThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Reservation Process : Ticket " + i + " Reserved");
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
// Thread using Runnable interface
class StatusThread implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Status Check : Ticket " + i + " Confirmed");
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
```

```
            System.out.println(e);
        }
    }
  }
}
// Main Class
public class Main {
  public static void main(String[] args) {
    // Thread using Thread class
    ReservationThread t1 = new ReservationThread();

    // Thread using Runnable interface
    Thread t2 = new Thread(new StatusThread());

    t1.start();
    t2.start();
  }
}
```

Output

```
Reservation Process : Ticket 1 Reserved
Status Check : Ticket 1 Confirmed
Reservation Process : Ticket 2 Reserved
Status Check : Ticket 2 Confirmed
Reservation Process : Ticket 3 Reserved
Status Check : Ticket 3 Confirmed
```

Reservation Process : Ticket 4 Reserved

Status Check : Ticket 4 Confirmed

Reservation Process : Ticket 5 Reserved

Status Check : Ticket 5 Confirmed

Result

Thus, a multithreaded Java application using the Thread class and Runnable interface was developed successfully.

Viva Questions

1. What is a thread?
2. What is multithreading?
3. What is the difference between extending the Thread class and implementing the Runnable interface?
4. Which approach is preferred and why?
5. What is the purpose of the start() method?

10. Design and Implementation of Synchronized Multithreaded Applications with Inter-thread Communication

Aim

To design and implement synchronized multithreaded applications using synchronization and inter-thread communication.

Program

```
// Shared Resource
class Buffer {
    private int data;
    private boolean available = false;
    // Producer Method
    public synchronized void produce(int value) {
        try {
            while (available) {
                wait();
            }
            data = value;
            System.out.println("Produced : " + data);
            available = true;
            notify();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
    // Consumer Method
    public synchronized void consume() {
```

```
        try {
            while (!available) {
                wait();
            }
            System.out.println("Consumed : " + data);
            available = false;
            notify();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

// Producer Thread
class Producer extends Thread {
    Buffer buffer;
    Producer(Buffer buffer) {
        this.buffer = buffer;
    }
    public void run() {
        for (int i = 1; i <= 5; i++) {
            buffer.produce(i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
```

```
                System.out.println(e);
            }
        }
    }
}

// Consumer Thread
class Consumer extends Thread {
   Buffer buffer;
    Consumer(Buffer buffer) {
        this.buffer = buffer;
    }
    public void run() {
        for (int i = 1; i <= 5; i++) {
            buffer.consume();
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
// Main Class
public class Main {
```

```
    public static void main(String[] args) {
        Buffer buffer = new Buffer();
        Producer producer = new Producer(buffer);
        Consumer consumer = new Consumer(buffer);
        producer.start();
        consumer.start();
    }
}
```

Output

```
Produced : 1
Consumed : 1
Produced : 2
Consumed : 2
Produced : 3
Consumed : 3
Produced : 4
Consumed : 4
Produced : 5
Consumed : 5
```

Result

Thus, synchronized multithreading with inter-thread communication was implemented successfully.

Viva Questions

1. What is synchronization?
2. Why is synchronization required?
3. What are race conditions?
4. Explain wait(), notify(), and notifyAll().
5. What is a synchronized method?

11. Development of Applications Demonstrating Thread Lifecycle Management and Deadlock Prevention Techniques

Aim

To develop Java applications demonstrating thread lifecycle management and deadlock prevention techniques.

Program

```
// Shared Resource
class Printer {

    public synchronized void printDocument(String threadName) {

        System.out.println(threadName + " is printing the document...");

        try {
            // Thread enters TIMED_WAITING state
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println(threadName + " has finished printing.");
    }
}
// Thread Class
class UserThread extends Thread {
```

```
    Printer printer;

    UserThread(Printer printer, String name) {
        super(name);
        this.printer = printer;
    }

    @Override
    public void run() {

        System.out.println(getName() + " started.");

        printer.printDocument(getName());

        System.out.println(getName() + " terminated.");
    }
}
// Main Class
public class Main {
   public static void main(String[] args) throws InterruptedException {
        Printer printer = new Printer();
        UserThread t1 = new UserThread(printer, "Thread-1");
        UserThread t2 = new UserThread(printer, "Thread-2");
        // NEW State
        System.out.println("Thread-1 State : " + t1.getState());
```

```
        // Runnable State
        t1.start();
        t2.start();
        Thread.sleep(200);
        System.out.println("Thread-1 Current State : " + t1.getState());
        // Wait for both threads to complete
        t1.join();
       t2.join();
        // TERMINATED State
        System.out.println("Thread-1 Final State : " + t1.getState());
        System.out.println("\nDeadlock Prevention:");
        System.out.println("Both threads access the shared printer");
        System.out.println("using a single synchronized method.");
        System.out.println("Since there is only one shared lock,");
        System.out.println("deadlock is prevented.");
    }
}
```

Output

```
Thread-1 State : NEW
Thread-1 started.
Thread-2 started.
Thread-1 is printing the document...
Thread-1 Current State : TIMED_WAITING


Thread-1 has finished printing.
```

Thread-1 terminated.

Thread-2 is printing the document...

Thread-2 has finished printing.

Thread-2 terminated.

Thread-1 Final State : TERMINATED

Deadlock Prevention:

Both threads access the shared printer

using a single synchronized method.

Since there is only one shared lock,

deadlock is prevented.

Result

Thus, thread lifecycle management and deadlock prevention techniques were demonstrated successfully.

Viva Questions

1. Explain the lifecycle of a thread.
2. What is deadlock?
3. How can deadlock be prevented?
4. What is thread scheduling?
5. What is thread priority?

12. Implementation of Generic Collection-Based Applications Using List, Set, Map, and Queue Interfaces

Aim

To implement generic collection-based applications using List, Set, Map, and Queue interfaces.

Program

```
import java.util.*;


public class Main {


   public static void main(String[] args) {


      // List Interface
      List<String> studentList = new ArrayList<>();
      studentList.add("Arun");
      studentList.add("Divya");
      studentList.add("Karthik");
      studentList.add("Divya");   // Duplicate Allowed


      System.out.println("Student List");
      System.out.println(studentList);


      // Set Interface
      Set<String> courseSet = new HashSet<>();
      courseSet.add("Java");
      courseSet.add("Python");
```

```
courseSet.add("Java");
courseSet.add("C++");

System.out.println("\nCourse Set");
System.out.println(courseSet);

// Map Interface
Map<Integer, String> studentMap = new HashMap<>();
studentMap.put(101, "Arun");
studentMap.put(102, "Divya");
studentMap.put(103, "Karthik");

System.out.println("\nStudent Map");
for (Map.Entry<Integer, String> entry : studentMap.entrySet()) {
   System.out.println("Roll No : " + entry.getKey()
         + "  Name : " + entry.getValue());
}

// Queue Interface
Queue<String> queue = new LinkedList<>();

queue.offer("Student-1");
queue.offer("Student-2");
queue.offer("Student-3");
```

```
        System.out.println("\nQueue Before Processing");
        System.out.println(queue);

        System.out.println("Processing : " + queue.poll());

        System.out.println("Queue After Processing");
        System.out.println(queue);
    }
}
```

Sample Output

```
Student List
[Arun, Divya, Karthik, Divya]

Course Set
[Java, Python, C++]

Student Map
Roll No : 101  Name : Arun
Roll No : 102  Name : Divya
Roll No : 103  Name : Karthik

Queue Before Processing
[Student-1, Student-2, Student-3]
```

Processing : Student-1

Queue After Processing

[Student-2, Student-3]

Result

Thus, generic collection-based applications using List, Set, Map, and Queue interfaces were implemented successfully.

Viva Questions

1. What is the Java Collections Framework?
2. Differentiate List and Set.
3. What is the difference between HashMap and TreeMap?
4. What is the purpose of Queue?
5. What are Generics in Java?

13. Design of Applications Utilizing Comparator and Comparable Interfaces for Customized Sorting Operations

Aim

To design Java applications utilizing Comparable and Comparator interfaces for customized sorting operations.

Program

```
import java.util.*;


// Student class implementing Comparable
class Student implements Comparable<Student> {


    int rollNo;
    String name;


    Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }


    // Default Sorting by Roll Number
    @Override
    public int compareTo(Student s) {
        return this.rollNo - s.rollNo;
    }


    @Override
```

```
    public String toString() {
        return rollNo + " - " + name;
    }
}

// Comparator for Sorting by Name
class NameComparator implements Comparator<Student> {

    @Override
    public int compare(Student s1, Student s2) {
        return s1.name.compareTo(s2.name);
    }
}

public class Main {

    public static void main(String[] args) {

        ArrayList<Student> students = new ArrayList<>();

        students.add(new Student(103, "Rahul"));
        students.add(new Student(101, "Anitha"));
        students.add(new Student(102, "Karthik"));

        System.out.println("Students Before Sorting");
```

```
        for (Student s : students)
            System.out.println(s);

        // Comparable Sorting
        Collections.sort(students);

        System.out.println("\nSorted by Roll Number (Comparable)");
        for (Student s : students)
            System.out.println(s);

        // Comparator Sorting
        Collections.sort(students, new NameComparator());

        System.out.println("\nSorted by Name (Comparator)");
        for (Student s : students)
            System.out.println(s);
    }
}
```

Output

```
Students Before Sorting
103 - Rahul
101 - Anitha
102 - Karthik
Sorted by Roll Number (Comparable)
101 - Anitha
```

102 - Karthik

103 - Rahul

Sorted by Name (Comparator)

101 - Anitha

102 - Karthik

103 - Rahul

Result

Thus, customized sorting using Comparable and Comparator interfaces was implemented successfully.

Viva Questions

1. What is Comparable?
2. What is Comparator?
3. What is the difference between Comparable and Comparator?
4. Which method is overridden in Comparable?
5. Why is Comparator preferred for multiple sorting criteria?

14. Development of File Processing Applications Using Java I/O Streams, Readers, Writers, and Buffered Streams

Aim

To develop Java applications for file processing using Java I/O streams, readers, writers, and buffered streams.

Program

```
import java.io.*;

public class Main {

   public static void main(String[] args) {

      String fileName = "student.txt";

      try {

         // Writing to the file using FileWriter and BufferedWriter

         FileWriter fw = new FileWriter(fileName);

         BufferedWriter bw = new BufferedWriter(fw);

         bw.write("Student Record");

         bw.newLine();

         bw.write("------------------------");

         bw.newLine();

         bw.write("Roll No : 101");

         bw.newLine();

         bw.write("Name    : Rahul");

         bw.newLine();

         bw.write("Course  : B.Tech Information Technology");

         bw.newLine();

         bw.write("CGPA    : 8.75");
```

```
            bw.close();
            System.out.println("Data written successfully.\n");
            // Reading from the file using FileReader and BufferedReader
            FileReader fr = new FileReader(fileName);
            BufferedReader br = new BufferedReader(fr);
            String line;
            System.out.println("Contents of the File");
            System.out.println("----------------------------");

            while ((line = br.readLine()) != null) {
               System.out.println(line);
            }
            br.close();
        } catch (IOException e) {
            System.out.println("Error : " + e.getMessage());
        }
    }
}
```

Output

```
Data written successfully.
Contents of the File
----------------------------
Student Record
------------------------
Roll No : 101
```

Name : Rahul

Course : B.Tech Information Technology

CGPA : 8.75

Result

Thus, file processing using Java I/O streams and buffered streams was implemented successfully.

Viva Questions

1. What is Java I/O?
2. Differentiate byte streams and character streams.
3. What is BufferedReader?
4. What is BufferedWriter?
5. Why are buffered streams faster?

15. Implementation of Object Persistence Mechanisms Using Serialization and Deserialization Techniques

Aim

To implement object persistence mechanisms using serialization and deserialization techniques in Java.

Program

```
import java.io.*;


// Student class implementing Serializable

class Student implements Serializable {


    private static final long serialVersionUID = 1L;


    int rollNo;

    String name;

    double cgpa;


    Student(int rollNo, String name, double cgpa) {

        this.rollNo = rollNo;

        this.name = name;

        this.cgpa = cgpa;

    }


    void display() {

        System.out.println("Student Details");

        System.out.println("------------------------");
```

```
        System.out.println("Roll No : " + rollNo);
        System.out.println("Name    : " + name);
        System.out.println("CGPA    : " + cgpa);
    }
}

public class Main {

    public static void main(String[] args) {

        Student student = new Student(101, "Rahul", 8.75);

        try {

            // Serialization
            FileOutputStream fos = new FileOutputStream("student.ser");
            ObjectOutputStream oos = new ObjectOutputStream(fos);

            oos.writeObject(student);

            oos.close();

            System.out.println("Object Serialized Successfully.\n");

            // Deserialization
```

```
            FileInputStream fis = new FileInputStream("student.ser");
            ObjectInputStream ois = new ObjectInputStream(fis);

            Student s = (Student) ois.readObject();

            ois.close();

            System.out.println("Object Deserialized Successfully.\n");

            s.display();

        } catch (IOException e) {

            System.out.println("I/O Error : " + e.getMessage());

        } catch (ClassNotFoundException e) {

            System.out.println("Class Not Found : " + e.getMessage());
        }
    }
}
```

Output

Object Serialized Successfully.

Object Deserialized Successfully.

Student Details

Roll No : 101

Name : Rahul

CGPA : 8.75

Result

Thus, object persistence using serialization and deserialization techniques was implemented successfully.

Viva Questions

1. What is serialization?
2. What is deserialization?
3. What is the Serializable interface?
4. What is the purpose of serialVersionUID?
5. What are the advantages of serialization?